Statistics 6910, Section 004, Fall 2013, Quiz 1 (25 Points)

Monday, October 14, 2013

Your Name: _____

Question 1: Function Writing (**13 Points**)

We often need to transform data via a log or reciprocal ($1/x$) transformation.

Write a function `TransformData` that has two arguments: `x` contains the data. `type` is optional with 1 as a default value. `type` should be interpreted as follows: If `type = 1`, we calculate the log with base 10 of `x`. If `type = 2`, we calculate `1 / x`. Otherwise, we produce a warning message that `type` must be one of the numbers 1 or 2 and the function returns a single `NA`.

There is **no need** for checking the class of `x`. If we can't perform a calculation for a particular class (e.g., if `x` is character vector), then let R report a default error message.

However, we want to control the behavior if we take the log of a number ≤ 0 or the reciprocal of 0. For the log, assign 999 if `x` is ≤ 0, and for the reciprocal ($1/x$), also assign 999 if `x` is $= 0$.

Don't laugh, but assigning 999 is a common way to mark missing values for climate data. See this NOAA web page as an example: `http://www.aoml.noaa.gov/hrd/format/asdl.html`. (It's not furloughed. ☺

Keep in mind that `x` can be a vector! Your function should return a vector of the same length as `x` in case `type` is 1 or 2 (otherwise, return `NA` and print a warning).

As a reminder, the log with base 10 can be calculated via `log10` and the help page for the ifelse function provides the following information:

```
ifelse {base} R Documentation
Conditional Element Selection

Description
ifelse returns a value with the same shape as test which is filled
with elements selected from either yes or no depending on whether
the element of test is TRUE or FALSE.

Usage
ifelse(test, yes, no)

Arguments
test  an object which can be coerced to logical mode.
yes   return values for true elements of test.
no    return values for false elements of test.
```

My function is given below:

The following output should be produced:

```
> TransformData(c(-10, 0, 10, 100, 1000))
[1] 999 999   1   2   3
> TransformData(c(-10, 0, 10, 100, 1000), type = 1)
[1] 999 999   1   2   3
> TransformData(c(-10, 0, 10, 100, 1000), type = 2)
[1]  -0.100 999.000   0.100   0.010   0.001
> TransformData(c(-10, 0, 10, 100, 1000), type = 3)
[1] NA
Warning message:
  Warning: type must be 1 or 2.
```

Question 2: Environments and Scope (**12 Points**)

We run the code below in an otherwise empty workspace. Indicate what each of the `cat` expressions is going to print. Warning: This is very poor R code. Read carefully...

Note the following from Adler (2010), p. 100: "The parent environment of a function is the environment in which the function was created."

```
f1 = function(i) {
  k = i * 5
  cat("#1:", "i=", i, "j=", j, "k=", k, "\n")
  i = 100
  return(k)
}

f2 = function(i) {
  j = 20
  k = f1(i)
  cat("#2:", "i=", i, "j=", j, "k=", k, "\n")
  i = 200
  return(k)
}

i = 5
j = 10
k = 15

k = f2(i)
cat("#3:", "i=", i, "j=", j, "k=", k, "\n")

for (i in 1:10)
  j = i + 1

cat("#4:", "i=", i, "j=", j, "k=", k, "\n")
```

As a result, the following text is printed:

```
#1: i=          j=          k=


#2: i=          j=          k=


#3: i=          j=          k=


#4: i=          j=          k=
```